

LAB EXPERIMENT – 1

8-Puzzle Problem using Python

Aim

To write a Python program to solve the **8-Puzzle problem** using the **A* search algorithm**.

Procedure

1. Represent the 8-puzzle as a **3 × 3 matrix** containing numbers from 1 to 8 and a blank space (0).
2. Define the **initial state** and **goal state**.
3. Generate all possible moves by moving the blank tile **up, down, left, or right**.
4. Calculate the **Manhattan distance** heuristic for each state.
5. Use the **A* algorithm** to select the state with the lowest cost.
6. Continue generating states until the **goal state** is reached.
7. Display the sequence of states from the initial state to the goal state.

Source Code

main.py

Visualize

Run ▶

```
1 import heapq
2
3 def manhattan(state, goal):
4     distance = 0
5
6     for i in range(9):
7         if state[i] != 0:
8             current_row = i // 3
9             current_col = i % 3
10
11             goal_index = goal.index(state[i])
12             goal_row = goal_index // 3
13             goal_col = goal_index % 3
14
15             distance += abs(current_row - goal_row)
16             distance += abs(current_col - goal_col)
17
18     return distance
19
20
21 def get_neighbors(state):
22     neighbors = []
23     zero = state.index(0)
24
25     row = zero // 3
26     col = zero % 3
27
28     moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
29
30     for dr, dc in moves:
31         new_row = row + dr
32         new_col = col + dc
33
34         if 0 <= new_row < 3 and 0 <= new_col < 3:
35             new_zero = new_row * 3 + new_col
36
37             new_state = list(state)
38             new_state[zero], new_state[new_zero] = \
39                 new_state[new_zero], new_state[zero]
40
41             neighbors.append(tuple(new_state))
42
43     return neighbors
44
45
46 def solve_puzzle(start, goal):
47     priority_queue = []
48
49     h = manhattan(start, goal)
50
51     heapq.heappush(priority_queue, (h, 0, start, []))
52
53     visited = set()
54
55     while priority_queue:
56         f, g, state, path = heapq.heappop(priority_queue)
57
58         if state in visited:
59             continue
```

Solution found!

Number of moves: 2

Steps:

Step 0

(1, 2, 3)

(4, 0, 6)

(7, 5, 8)

Step 1

(1, 2, 3)

(4, 5, 6)

(7, 0, 8)

Step 2

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

main.py	Visualize	Run ▶	Output
<pre>49 h = manhattan(start, goal) 50 51 heapq.heappush(priority_queue, (h, 0, start, [])) 52 53 visited = set() 54 55 while priority_queue: 56 f, g, state, path = heapq.heappop(priority_queue) 57 58 if state in visited: 59 continue 60 61 visited.add(state) 62 63 new_path = path + [state] 64 65 if state == goal: 66 return new_path 67 68 for neighbor in get_neighbors(state): 69 if neighbor not in visited: 70 new_g = g + 1 71 new_f = new_g + manhattan(neighbor, goal) 72 73 heapq.heappush(74 priority_queue, 75 (new_f, new_g, neighbor, new_path) 76) 77 78 return None 79 80 81 def print_state(state): 82 for i in range(0, 9, 3): 83 print(state[i:i+3]) 84 print() 85 86 87 # Initial and goal states 88 start = (1, 2, 3, 89 4, 0, 6, 90 7, 5, 8) 91 92 goal = (1, 2, 3, 93 4, 5, 6, 94 7, 8, 0) 95 96 solution = solve_puzzle(start, goal) 97 98 if solution: 99 print("Solution found!") 100 print("Number of moves:", len(solution) - 1) 101 print("\nSteps:") 102 for step, state in enumerate(solution): 103 print("Step", step) 104 print_state(state) 105 else: 106 print("No solution exists.") 107</pre>			<pre>Solution found! Number of moves: 2 Steps: Step 0 (1, 2, 3) (4, 0, 6) (7, 5, 8) Step 1 (1, 2, 3) (4, 5, 6) (7, 0, 8) Step 2 (1, 2, 3) (4, 5, 6) (7, 8, 0)</pre>

Result

Thus, the 8-Puzzle problem was successfully solved using the A* search algorithm in Python.